

Recursion Mastery Problem Set (Basic → Advanced)

This document contains a structured recursion roadmap designed to build strong recursive thinking. The problems are grouped from beginner to advanced level. Solve them in order and focus on:

- Base cases
- Recursive hypothesis
- Recursive tree visualization
- Backtracking patterns
- Divide & conquer thinking
- Transition from recursion → dynamic programming

Level 1 — Recursion Fundamentals

- 1. Print numbers from 1 to N
- 2. Print numbers from N to 1
- 3. Sum of first N natural numbers
- 4. Factorial of a number
- 5. Power function (a^b)
- 6. Sum of digits
- 7. Reverse a string
- 8. Check palindrome string
- 9. Check sorted array
- 10. Find maximum element in array
- 11. Find minimum element in array
- 12. Linear search using recursion
- 13. Binary search using recursion
- 14. Count zeros in a number
- 15. Product of digits
- 16. Count digits in number

Level 2 — Intermediate Array/String Recursion

- 17. Remove consecutive duplicates from string
- 18. Move all X to end of string
- 19. Generate all subsequences
- 20. Generate all substrings
- 21. Print all subsets of array
- 22. Subset sum recursion

- 23. Find all occurrences of target
- 24. Check if array is palindrome
- 25. Replace pi with 3.14
- 26. String permutations
- 27. Phone keypad combinations
- 28. Decode ways recursion
- 29. Generate balanced parentheses
- 30. Generate binary strings

Level 3 — Backtracking Foundation

- 31. Rat in a maze
- 32. N-Queens problem
- 33. Sudoku solver
- 34. Knight's tour
- 35. Word search
- 36. Combination Sum
- 37. Combination Sum II
- 38. Palindrome partitioning
- 39. Partition array into K subsets
- 40. Restore IP addresses
- 41. Generate all permutations
- 42. Letter case permutation
- 43. M-coloring problem
- 44. Crossword puzzle solver

Level 4 — Advanced Recursive Thinking

- 45. Tower of Hanoi
- 46. Josephus problem
- 47. K-th symbol in grammar
- 48. Recursive stack sorting
- 49. Delete middle element of stack
- 50. Reverse stack using recursion
- 51. Recursive queue reversal
- 52. Recursive linked list reversal
- 53. Merge sort
- 54. Quick sort

- 55. Generate Gray code
- 56. Recursive expression evaluation
- 57. Catalan recursion problems

Level 5 — Recursion + Dynamic Programming Bridge

- 58. Fibonacci recursion → memoization
- 59. Climbing stairs
- 60. House robber
- 61. 0/1 Knapsack
- 62. Subset sum
- 63. Equal partition
- 64. Coin change
- 65. Target sum
- 66. Longest common subsequence
- 67. Longest increasing subsequence
- 68. Matrix chain multiplication
- 69. Palindrome partitioning DP
- 70. Boolean parenthesization
- 71. Egg dropping problem
- 72. Burst balloons

Level 6 — Interview-Level Advanced Problems

- 73. Word Break
- 74. Wildcard matching
- 75. Regular expression matching
- 76. Scrambled string
- 77. Different ways to add parentheses
- 78. Expression add operators
- 79. Remove invalid parentheses
- 80. Partition to K equal sum subsets
- 81. Cherry pickup
- 82. Digit DP introduction
- 83. Tree DP recursion
- 84. Bitmask recursion
- 85. Memoized DFS on graphs

How to Practice Recursion Effectively

1. Always identify the smallest solvable subproblem.
2. Write the base case before the recursive call.
3. Dry-run recursion trees on paper.
4. Learn these major patterns:
 - Pick / Not Pick
 - Backtracking
 - Divide & Conquer
 - Recursive DFS
 - Recursion + Memoization
5. Once recursion becomes easy, dynamic programming becomes much easier.